UDAIPUR CROWDWISE

Hackathon Project Guide

Problem statement
Predict crowded tourist destinations and recommend alternative places and better visiting times.

A practical blueprint for building a tourist-support application using destination data, crowd observations, weather and event information, a prediction engine, and alternative-place recommendations.

Core user journey
Choose a destination and time → estimate crowd level → discover a suitable alternative or a quieter time slot.

Recommended approach: start with a small, transparent MVP. Use verified place information and clearly labelled crowd estimates; add machine learning only when you have enough real observations to test it.

1. What are you building?

The application should answer three questions for tourists in Udaipur:

• **Where might it be crowded?** Estimate whether a destination is likely to be low, moderately, or highly crowded at a selected time.

• **Where can I go instead?** Suggest other attractions that fit the visitor's interests, travel distance, opening hours, accessibility, and predicted crowd level.

• **When should I visit?** Recommend a less busy time slot based on historical patterns, observations, weather, weekdays, holidays, and events.

Important: Do not claim to know the exact number of visitors unless you have a reliable measurement source. A predicted crowd category is not the same as a live headcount.

2. Data collection plan

You do not need to collect everything yourself. Combine public destination information, weather and event data, volunteer observations, and user-submitted crowd reports.

A. Tourist destinations and place details

Start with 15–30 Udaipur attractions. Create a CSV or MongoDB collection with place name, category, coordinates, address, opening hours, official source, accessibility notes, and suitable visitor interests.

• Official Rajasthan Tourism website: https://www.tourism.rajasthan.gov.in/udaipur.html

• Google Places API: place details and nearby attractions; check current billing, usage limits, and field-specific pricing.

• OpenStreetMap: geographic and map data, subject to the relevant data and tile-provider policies.

• Your own verified CSV/JSON file is the simplest starting point for a hackathon prototype.

Place	Category	Information to store
City Palace	Heritage	Coordinates, opening hours, source
Lake Pichola	Lake / boating	Coordinates, access details, source
Fateh Sagar Lake	Lake / waterfront	Coordinates, access details, source
Ahar Museum and Cenotaphs	History	Coordinates, opening hours, source
Saheliyon-ki-Bari	Garden	Coordinates, opening hours, source

B. Crowd data — the most important part

Method 1: Manual observations. Ask team members or volunteers to observe a defined area at consistent intervals, such as every 30 minutes. Record an approximate count for that area or use a consistent low / moderate / high rating.

Method 2: Visitor reports. Add a "Report crowd level" form. Store the selected place, timestamp, rating, and optional anonymous session identifier. Use multiple reports and show how many reports support the estimate.

Method 3: Historical visitor counts. Ask attraction managers, hotels, or local tour operators whether they can share anonymized historical counts. Get permission and do not assume that such data is available.

Method 4: Events and holidays. Collect public holidays, festivals, and local events from official or otherwise verifiable sources. These are useful signals, but they do not directly measure crowd size.

C. Weather data

Open-Meteo offers forecast and historical weather APIs: https://open-meteo.com/en/docs and https://open-meteo.com/en/docs/historical-weather-api. Consider temperature, precipitation, weather conditions, and time of day. Weather is a supporting signal, not a direct crowd measurement.

D. Suggested observation dataset

Field	Purpose
place_id	Unique destination identifier
observed_at	Date and time of the observation
crowd_level	Consistent label such as 1 = low, 2 = moderate, 3 = high
visitor_count	Optional count, only when measured using a defined method
temperature / rainfall	Weather context
is_weekend / is_holiday	Calendar features
event_nearby	Whether a relevant event is taking place
data_source / sample_count	Provenance and strength of supporting data

Example rows must be labelled as sample or synthetic data unless they come from real observations. With very little data, demonstrate a prototype but do not claim broad or seasonally accurate predictions.

3. Recommended technologies

Layer	Technology	Role
Frontend	React.js + Tailwind CSS	Search, filters, crowd cards, map, recommendations, responsive UI
Backend	Node.js + Express.js	API endpoints, data processing, prediction logic, recommendation ranking
Database	MongoDB	Places, observations, reports, events, predictions
Maps	Leaflet + OpenStreetMap	Show attractions and alternatives on a map
Weather	Open-Meteo API	Forecast and historical weather features
Places	Google Places API or OpenStreetMap	Place details and nearby attractions
Machine learning (optional)	Python + scikit-learn	Train and evaluate a classifier when enough real data exists
Hosting	Vercel/Netlify, Node-compatible host, MongoDB Atlas	Deploy the demo and database
Version control	GitHub	Team collaboration and code history

Recommendation: Since the project can be built with JavaScript, keep the main application in React, Node.js, Express, and MongoDB. Do not add a separate Python service unless the team has time and a validated ML use case.

4. How to predict crowd levels

Option A — Rule-based baseline (best for a fast MVP)

Create a score from 0 to 100 using normalized signals such as historical crowd level, weekend/holiday, nearby events, and weather. Start with example weights, then adjust them using real observations.

Signal	Illustrative weight
Historical crowd pattern	40%
Weekend / holiday	30%
Local events	20%
Weather conditions	10%

These weights are examples only, not scientifically validated Udaipur crowd patterns. Example categories might be 0–34 = low, 35–64 = moderate, and 65–100 = high. Tune thresholds and weights using observations; normalize each input before combining it.

Option B — Machine learning

If you have enough historical observations, train a Decision Tree or Random Forest classifier with features such as place, hour, weekday, holiday, event, weather, and recent crowd observations. Evaluate it on held-out dates or time periods. Avoid randomly splitting near-duplicate observations from the same period across training and test sets, because that can make performance look better than it is.

Expose the trained model through a small Python FastAPI service only if needed. Otherwise, a transparent rules-based baseline is a sensible first version.

5. Alternative destination recommendations

When a selected destination is expected to be crowded, the recommendation engine should:

• Find other attractions within a reasonable travel distance.

• Exclude destinations that are closed or unsuitable at the requested time.

• Match the visitor’s interests (heritage, gardens, lakes, museums, outdoor activities).

• Prefer alternatives with lower predicted crowd levels, while considering distance and accessibility.

• Offer a quieter time slot at the original destination as well as alternative places.

A simple ranking score can combine low predicted crowd, interest match, proximity, and open status. For example: Score = w1 × low-crowd suitability + w2 × interest match + w3 × proximity + w4 × open status. The weights should reflect your product priorities.

Candidate Udaipur alternatives to evaluate include Ahar Museum and Cenotaphs, Saheliyon-ki-Bari, and Fateh Sagar Lake. They must not be labelled “low crowd” unless your data supports that conclusion for the selected time. Use a routing service for travel-time estimates; straight-line distance is not driving or walking time.

6. Application architecture

React frontend
Search destination • Select date/time • Map • Crowd cards • Recommendations

Node.js + Express backend
API endpoints • Data processing • Recommendation ranking

Data and services
MongoDB • Weather API • Places/map data

Prediction engine
Rules-based baseline or validated ML model

Suggested API endpoints

Endpoint	Purpose
GET /api/places	List tourist destinations
GET /api/crowd/predict?placeId=P001&date;=2026-10-17&time;=11:00	Predict crowd level for a place and time
GET /api/alternatives?placeId=P001	Return suitable alternative destinations
POST /api/crowd/report	Submit a visitor crowd report
GET /api/weather	Retrieve weather information

These are example endpoints for your own backend, not existing third-party APIs.

7. Hackathon execution plan

Phase	Tasks	Deliverable
1. Foundation	Build 15–30 destination records; search page; map	Working destination explorer
2. Data	Integrate weather/calendar signals; crowd-report form; seed clearly labelled sample data	Usable data pipeline
3. Intelligence	Implement baseline score; rank alternatives; suggest time slots	End-to-end recommendation flow
4. Demo	Test mobile layout; show explanations and confidence; document limitations	Reliable hackathon demo

8. Features that can impress judges

• **Explainable predictions:** show the factors behind a high-crowd estimate.

• **Confidence indicators:** distinguish many supporting observations from sparse data.

• **Live crowd reporting:** let visitors submit reports to improve future estimates.

• **Time-aware alternatives:** suggest both another place and a quieter time to visit the original place.

• **Responsible recommendations:** avoid sending tourists to closed, unsafe, inaccessible, or environmentally sensitive sites.

9. Data quality, privacy, and honest claims

• Record where each dataset came from and when it was last updated.

• Respect API terms, rate limits, attribution requirements, and billing rules.

• Do not scrape protected crowd charts or imply access to a live-crowd API you do not have.

• Avoid collecting precise personal location or identifying information unless genuinely needed and properly consented to.

• Mark synthetic/sample data clearly in the interface and presentation.

• Validate predictions against later observations and report performance honestly.

10. Demo storyline

1. A tourist selects City Palace and a date/time.
2. The app shows a crowd estimate with a clear explanation and confidence indicator.
3. The app proposes a suitable alternative based on interests, opening hours, distance, and its estimated crowd level.
4. It also recommends a quieter time at the original attraction.
5. The presenter explains the data sources, how reports are collected, and what the model can and cannot claim.

Useful starting links

• Rajasthan Tourism — Udaipur: https://www.tourism.rajasthan.gov.in/udaipur.html

• Open-Meteo forecast API: https://open-meteo.com/en/docs

• Open-Meteo historical weather: https://open-meteo.com/en/docs/historical-weather-api

• Google Places API documentation: https://developers.google.com/maps/documentation/places/web-service

• OpenStreetMap: https://www.openstreetmap.org/

• Leaflet: https://leafletjs.com/

• scikit-learn: https://scikit-learn.org/stable/

Final recommendation

Build a complete, working flow before adding complexity: verified place data → crowd estimate → alternative place and time recommendation → user feedback. A transparent prototype with reliable sourcing and a thoughtful demo is stronger than an ambitious system that presents guesses as live facts.